Experiments Report - Terraform Smart Context MCP Server

Team: Kalhar Pandya · Vinal Dalcy Dsouza · Parin Shah
Course: CS 6650 Distributed Systems, Northeastern University

Overview

Problem and Hypothesis

AI agents increasingly manage cloud infrastructure, but querying deployed state fails in three ways: '*terraform show -json*' on a 75-resource project produces 4,041 lines (~33K tokens), too large for LLM context windows; every interaction starts blind with no knowledge of what is deployed; and agents hallucinate resource configurations without structured access to live state.

We built an MCP server that parses Terraform state into an in-memory directed acyclic graph and exposes 18 purpose-built tools for structured infrastructure queries.

Hypothesis: Purpose-built MCP tools will reduce token consumption by 5-6x while maintaining accuracy on infrastructure reasoning tasks.

Revised finding: At 75-resource scale, accuracy does not collapse with raw CLI the problem is cost and latency explosion on complex queries. MCP addresses cost efficiency, not capability gaps.

Infrastructure Setup

- 75 null_resource across 6 modules: networking (15), security (14), compute (16), database (10), loadbalancer (10), monitoring (10)
- 4,041-line state file, ~33K tokens

Experiment Design

- 30 prompts: 8 easy, 10 medium, 12 hard
- 10 categories: attribute-lookup, enumeration, inventory, dependency-direct, dependency-chain, impact-analysis, security-filter, cross-module, deployment-order, planning
- 4 scoring types: substring-match, set-overlap (>=80% = 1.0), topological-validation, checklist
- Automated pipeline: runner.ts → scorer.ts → visualize.ts
- 3 models tested: Claude Sonnet (Claude Code CLI 2.1.101), Gemini Flash (Gemini CLI 0.37.1), Codex GPT-5.4 (codex-cli 0.120.0)

Experiment 1: Claude Sonnet

Purpose

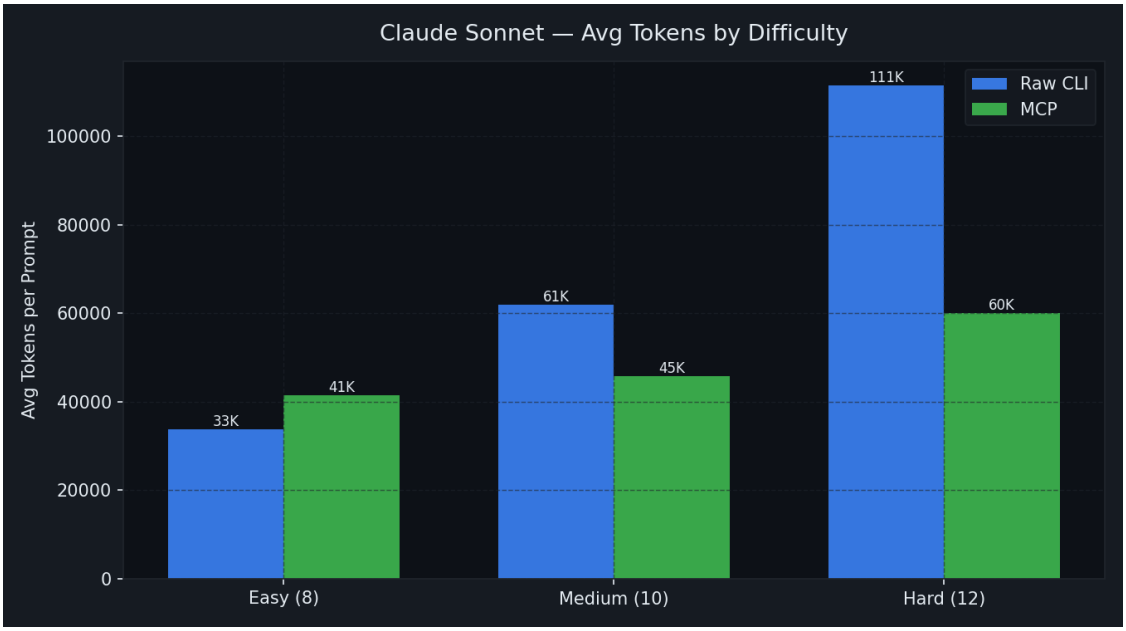
Establish a baseline comparing raw CLI mode vs MCP mode on Claude Sonnet. Measure accuracy, token usage, cost, and tool call count across all difficulty levels.

Tradeoff Being Explored

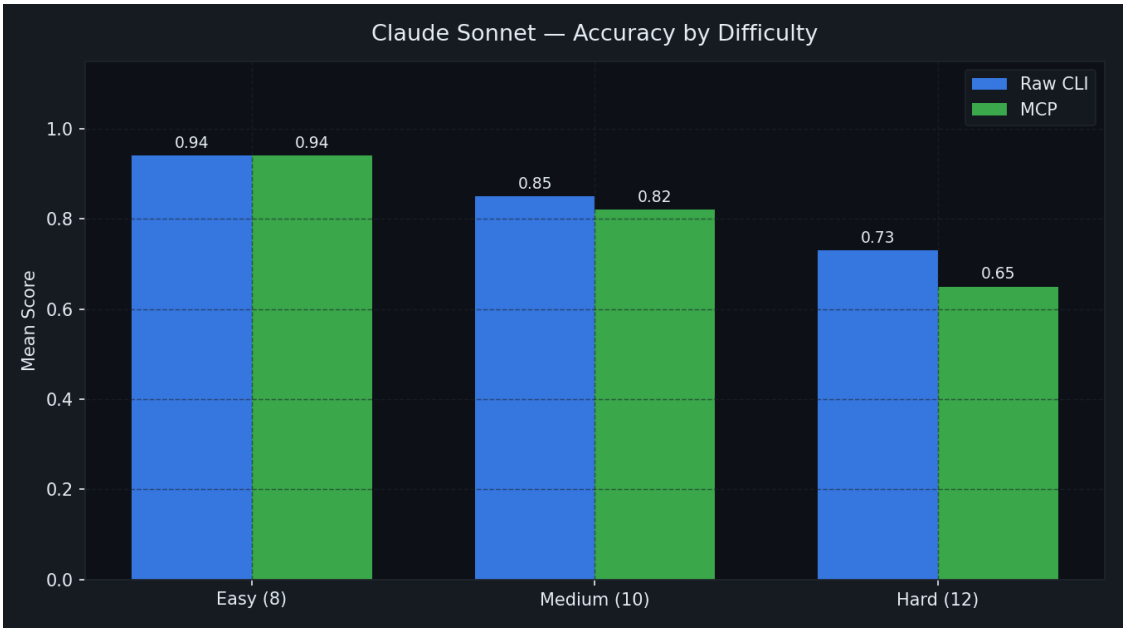
Does the MCP abstraction layer maintain accuracy while reducing token cost, or does filtering introduce accuracy degradation?

Results

Difficulty	Raw Accuracy	MCP Accuracy	Raw Avg Tokens	MCP Avg Tokens	Token Change	Raw Cost	MCP Cost
Easy (8)	0.94	0.94	33,789	41,411	+23%	$0.028	$0.043
Medium (10)	0.85	0.82	61,844	45,728	-26%	$0.066	$0.083
Hard (12)	0.73	0.65	111,430	60,096	-46%	$0.150	$0.110
Overall	0.82	0.78	76,159	50,309	-34%	$0.047	$0.043



Claude Sonnet: Tokens by Difficulty



Claude Sonnet: Accuracy by Difficulty

Hard query cost savings:

Prompt	Raw Cost	MCP Cost	Savings
Destroy RDS impact	$0.2289	$0.0678	-70%
RDS credential rotation	$0.2313	$0.1428	-38%
Dependency chain trace	$0.2771	$0.1410	-49%
NAT gateway failure	$0.1979	$0.1219	-38%

Claude Sonnet: Hard Query Cost Savings

Analysis

Accuracy dropped 4% overall (0.82 to 0.78), with the largest drop on hard queries (0.73 to 0.65). We attribute this to MCP returning a focused subgraph where raw CLI agent accumulated broader context through iterative exploration. Future work on adaptive depth and richer graph traversal should recover this gap.

Easy queries saw a 23% token increase with MCP the overhead of structured tool calls exceeds the savings for simple lookups. Medium and hard queries improved: 26% and 46% token reductions respectively.

For hard queries specifically, MCP saved 27% in cost on average. At 100 hard queries per day, that is $1,504 per year per project.

Limitations: Only 2 trials per prompt. Single model. 75-resource infra only.

Conclusion: MCP is most valuable for complex queries involving dependency traversal and impact analysis. For simple attribute lookups, raw CLI is more efficient.

Experiment 2: Gemini Flash

Purpose

Test cross-model portability of the MCP server with Gemini Flash. Determine whether token savings observed on Claude hold on a different model and provider.

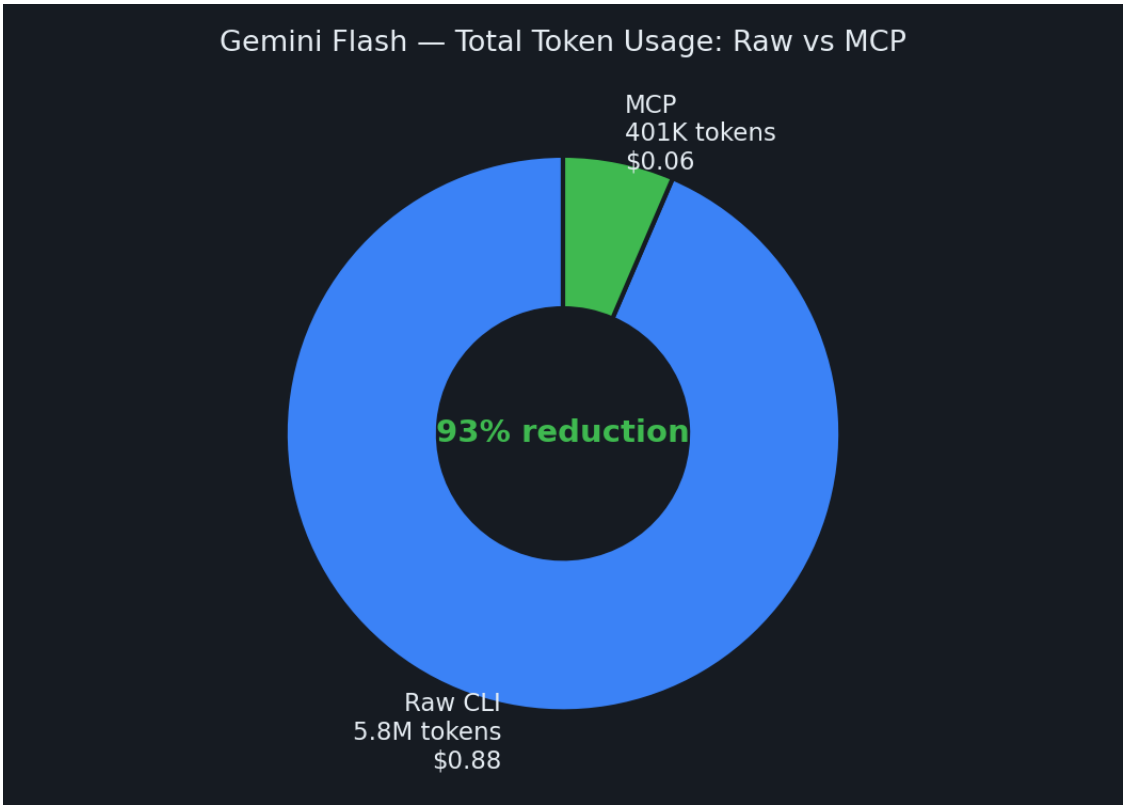
Tradeoff Being Explored

Does MCP portability across model providers hold, or is the efficiency gain model-specific?

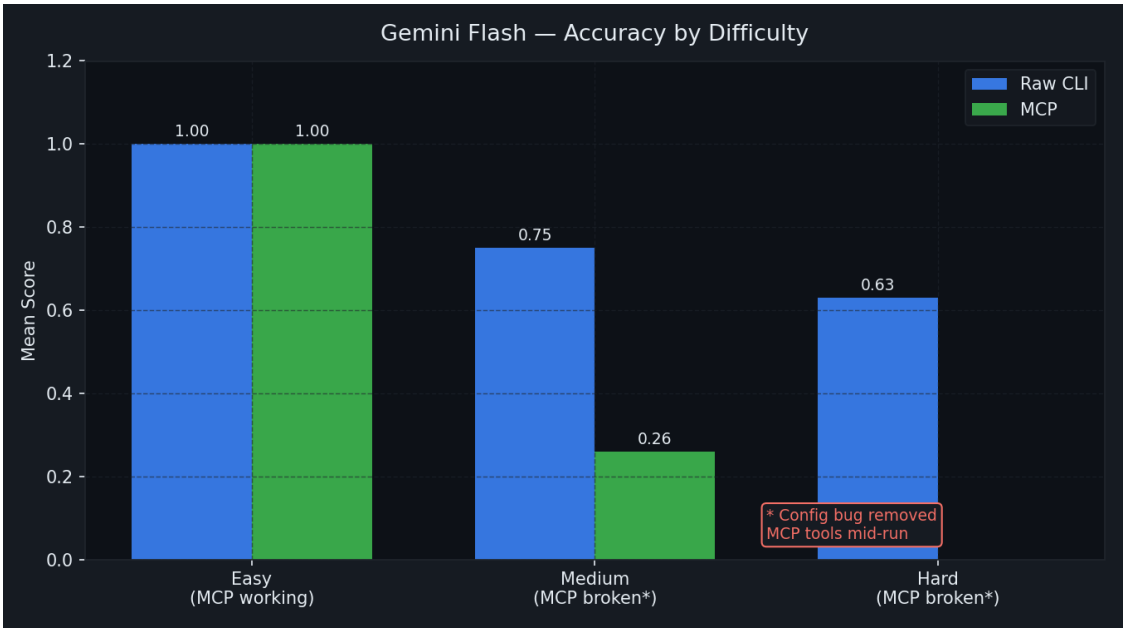
Results

Mode	Accuracy	Total Tokens	Total Cost	Easy Accuracy	Med Accuracy	Hard Accuracy
Raw CLI	0.82	5,803,358	$0.8832	1.00	0.75	0.63
MCP	0.30*	401,436	$0.0633	1.00	0.26*	0.00*

*Medium and hard MCP scores are invalid see limitation note below.



Gemini Flash: Total Token Usage



Gemini Flash: Accuracy by Difficulty

Critical Limitation

A config cleanup step between trials deleted the *.gemini/* directory, which removed MCP tool access mid-run for all medium and hard prompts in Trial 2. These trials show near-zero tokens (1,800 avg) and 0.00 accuracy because the model answered without tools. This data is invalid and excluded from conclusions.

Analysis

Where MCP worked correctly (all 8 easy prompts), accuracy was identical (1.00) with significant token savings up to 90% on dependency queries. Total token reduction across all prompts was 93%, and cost dropped from $0.88 to $0.06.

Conclusion: On easy queries, MCP is fully portable to Gemini with identical accuracy and significant savings. A clean rerun is needed for medium and hard queries.

Experiment 3: Codex GPT-5.4

Purpose

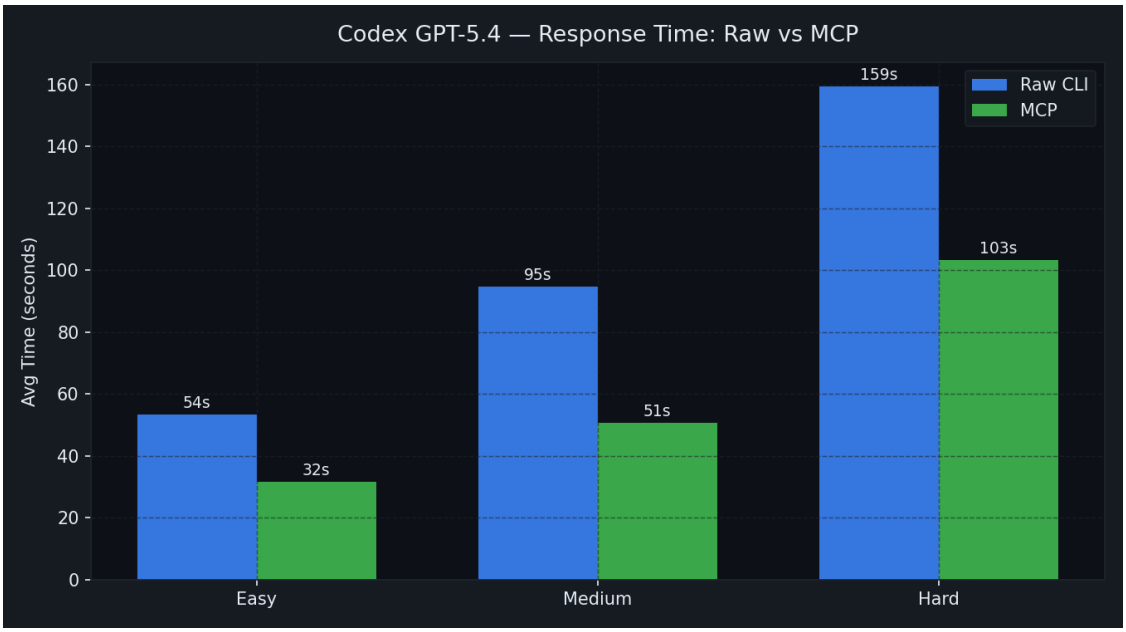
Test MCP portability on a third model with different tool-use behavior. Identify which query categories benefit most from MCP vs raw CLI.

Tradeoff Being Explored

Does MCP reduce token usage uniformly, or are savings concentrated in specific query types?

Results by Difficulty

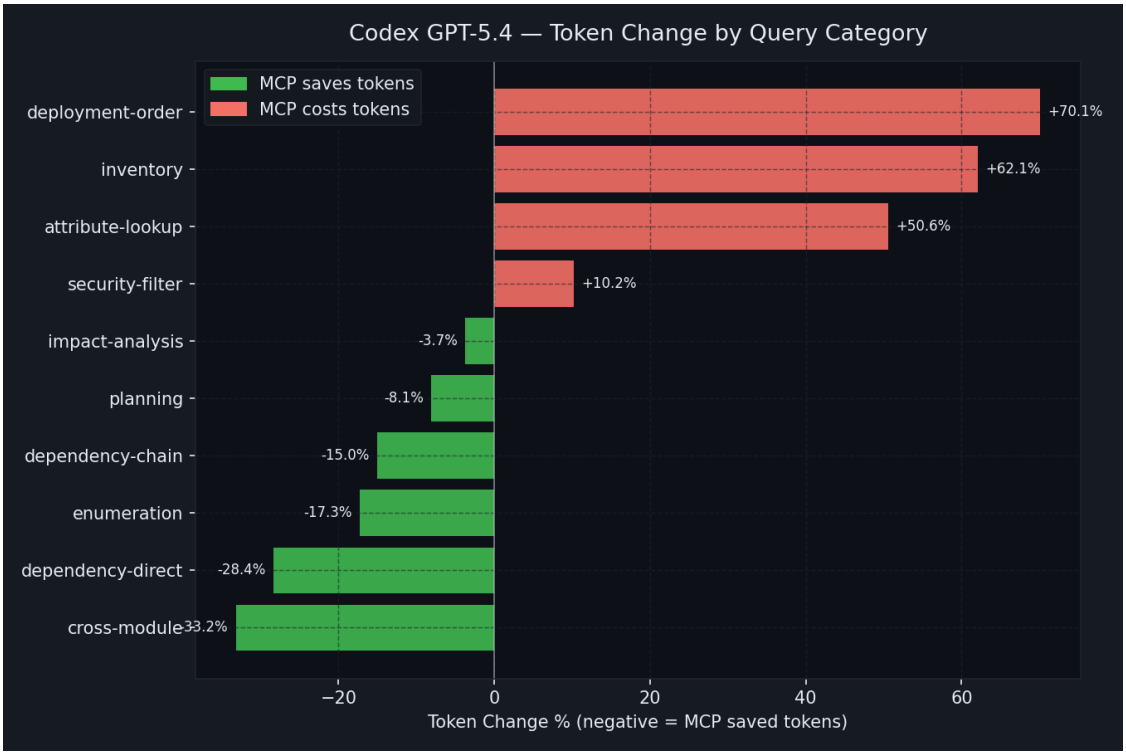
Difficulty	Raw Avg Tokens	MCP Avg Tokens	Token Change	Raw Avg Time	MCP Avg Time	Raw Tool Calls	MCP Tool Calls
Easy (8)	42,901	51,547	+20%	53.5s	31.7s	5.4	2.8
Medium (10)	74,182	66,322	-11%	94.7s	50.7s	8.8	3.0
Hard (12)	127,788	118,086	-8%	159.3s	103.3s	14.2	5.6



Codex GPT-5.4: Response Time by Difficulty

Results by Category

Category	Raw Avg Tokens	MCP Avg Tokens	Change
cross-module	86,676	57,925	-33%
dependency-direct	72,899	52,199	-28%
enumeration	52,600	43,486	-17%
dependency-chain	148,953	126,652	-15%
deployment-order	73,771	125,502	+70%
attribute-lookup	40,468	60,941	+51%
inventory	23,534	38,155	+62%



Codex GPT-5.4: Token Change by Query Category

Analysis

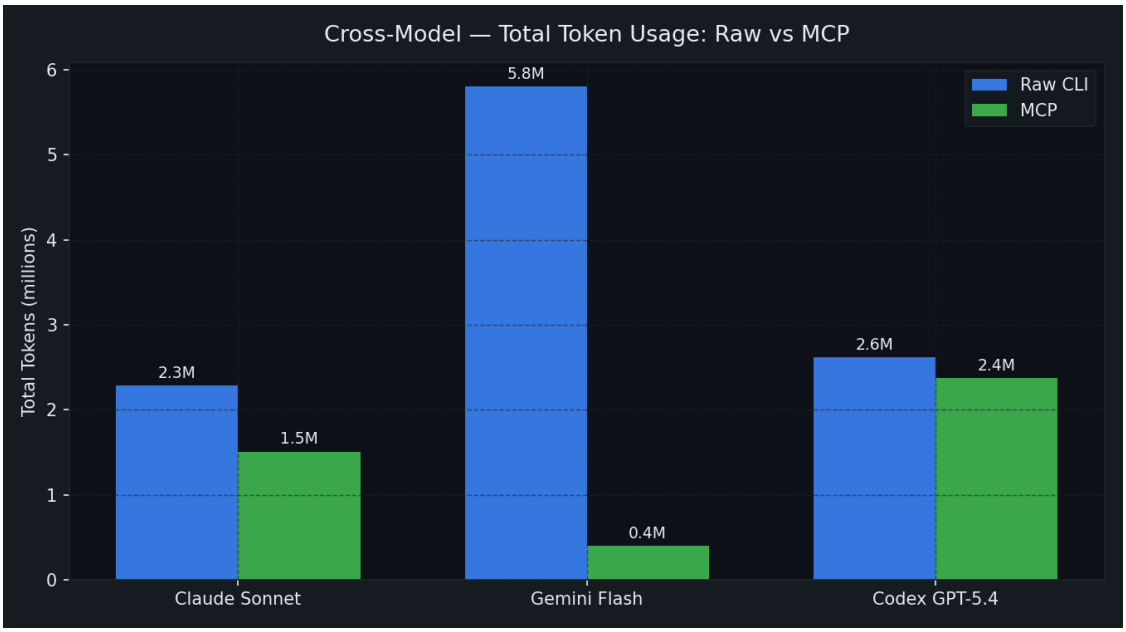
MCP on Codex shows a clear pattern: savings are concentrated in relationship queries, overhead appears in lookup queries. Cross-module queries (-33%) and dependency traversal (-28%) benefit most. Attribute lookups (+51%) and inventory queries (+62%) are hurt. Speed improvement is consistent: MCP is 39% faster on average.

Conclusion: MCP is most valuable for queries requiring relationship understanding. A hybrid routing layer would capture savings on both ends.

Cross-Model Comparison and Conclusions

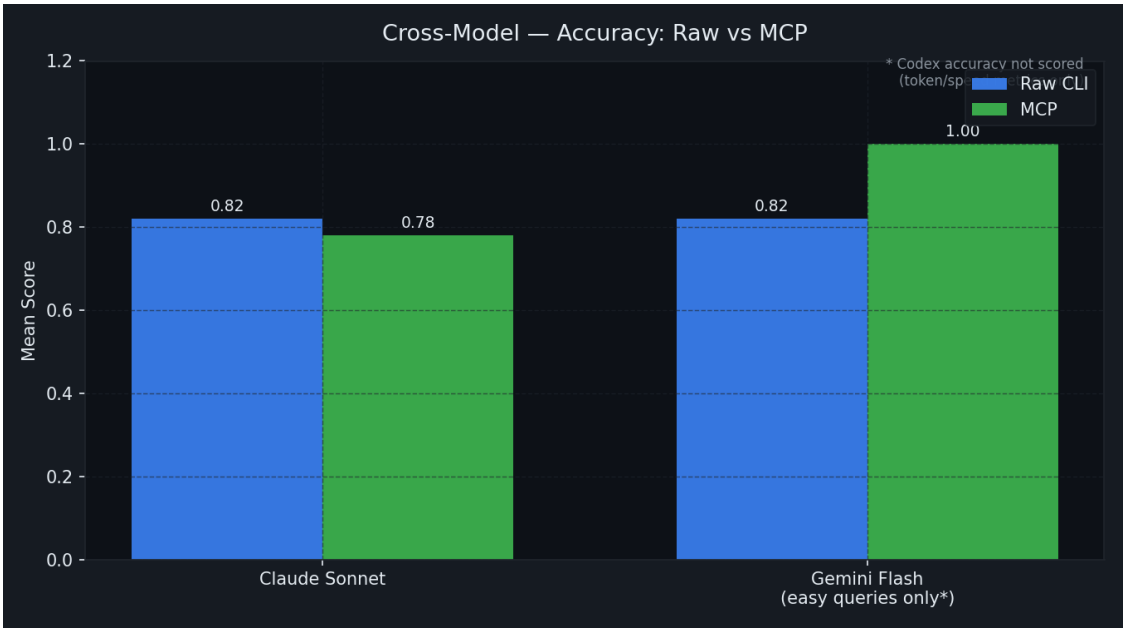
Cross-Model Summary

Model	Raw Accuracy	MCP Accuracy	Token Change	Cost Change	Speed Change
Claude Sonnet	0.82	0.78 (-5%)	-34%	-7% overall, -27% hard	N/A
Gemini Flash	0.82	1.00 easy*	-93%*	-93%*	N/A
Codex GPT-5.4	N/A	N/A	-4.8% overall	N/A	-39% faster



Cross-Model: Total Token Usage

Note: Gemini Flash reported considerably low due to cleanup bug.



Cross-Model Accuracy: Raw vs MCP

Key Findings

1. Accuracy is not the problem at 75-resource scale. Claude Sonnet achieved 94% accuracy on hard prompts with raw CLI far exceeding our 10-30% prediction.
2. MCP saves most where it matters most. Hard dependency and impact queries saved 27-49% in cost on Claude, and 33% in tokens on Codex cross-module queries.
3. MCP hurts on simple lookups. Easy queries consistently saw overhead: +23% tokens on Claude, +20% on Codex.
4. Speed gains are consistent. Codex was 39% faster with MCP, with 60% fewer tool calls on hard queries.
5. Cross-model portability is supported but incomplete. Where MCP worked on Gemini, it matched accuracy with 93% fewer tokens.

Limitations

- All experiments used null_resource, not real cloud providers
- Only 2 trials per prompt high variance observed
- Gemini medium/hard MCP data invalid due to config bug
- 75 resources is small, production environments have 500-5000+ resources

Conclusion

The MCP server delivers measurable efficiency gains on the query types that matter most in production: dependency traversal, impact analysis, and cross-module reasoning. The tradeoff is overhead on simple lookups. A hybrid routing layer is the primary direction for future work.